

DESARROLLO DE SOFTWARE

# **Trabajo Práctico Integrador**

Estructuras de Datos, Contrato Equals/HashCode y Complejidad Algorítmica

## **Integrantes:**

SINATRA FERNANDEZ, Thiago Gabriel  
CASTELLARIN, Franco Tomás  
CLAROS FERNANDEZ, Mailen Agustina  
OLIVA, Matías Gabriel  
VEGA, Karina Rocio

**2026 - 3K09**

---

## DESARROLLO DE SOFTWARE

# Trabajo Práctico Integrador

Estructuras de Datos, Contrato Equals/HashCode y Complejidad Algorítmica

## Consigna 1: Diagnóstico de Bug Silencioso y Análisis de Memoria

Un equipo de desarrollo junior implementó el siguiente sistema para gestionar el acceso de empleados a un edificio mediante tarjetas de identificación mutable. El código no lanza excepciones en tiempo de ejecución, pero presenta comportamientos anómalos en producción.

```
import java.util.*;

class TarjetaAcceso {
    private String codigo;
    private int nivelAcceso;

    public TarjetaAcceso(String codigo, int nivelAcceso) {
        this.codigo = codigo;
        this.nivelAcceso = nivelAcceso;
    }

    public void setNivelAcceso(int nivelAcceso) {
        this.nivelAcceso = nivelAcceso;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        TarjetaAcceso tarjeta = (TarjetaAcceso) o;
        return nivelAcceso == tarjeta.nivelAcceso && Objects.equals(codigo, tarjeta.codigo);
    }

    @Override
    public int hashCode() {
        return Objects.hash(codigo, nivelAcceso);
    }
}

public class ControlAcceso {

    public static void main(String[] args) {
        Set<TarjetaAcceso> tarjetasValidas = new HashSet<>();
        TarjetaAcceso t1 = new TarjetaAcceso("ACC-101", 2);

        tarjetasValidas.add(t1);

        // Modificación posterior a la inserción
        t1.setNivelAcceso(5);

        System.out.println("¿Tarjeta válida en sistema?: " + tarjetasValidas.contains(t1));
    }
}
```

```
        System.out.println("Cantidad de tarjetas registradas: " + tarjetasValidas.size());  
    }  
}
```

**Responder y Justificar:**

1. **Traza del Estado Interno:** Explique paso a paso qué ocurre dentro de la estructura HashSet:

**a. Al ejecutar `tarjetasValidas.add(t1)`.**

Se calcula el valor del `hashCode` del objeto *t1* basándose en los atributos *código* y *nivelAcceso* para almacenarlo. En este instante, estos valores están definidos en "ACC-101" y 2 respectivamente.

**b. Al ejecutar `t1.setNivelAcceso(5)`.**

Cambia el atributo *nivelAcceso* de *t1* de 2 a 5. Como consecuencia, al generar su `hashCode()` basándose en la nueva combinación ("ACC-101" y 5), se obtendrá un valor totalmente distinto al calculado cuando se creó el objeto.

**c. Al ejecutar `tarjetasValidas.contains(t1)`.**

Calcula el `hashCode()` de *t1* nuevamente pero ahora con el nivel de acceso 5, entonces busca el `hashCode` nuevo en un bucket distinto del bucket real del objeto, por ello pierde el objeto y devuelve false.

2. **Explicación del Cajón (Bucket):** ¿En qué número/ubicación de cajón quedó guardado el objeto y en cuál lo busca Java al hacer `contains()`?

El objeto quedó guardado, en el bucket correspondiente a la combinación ("ACC-101", 2). Sin embargo, al realizar la operación `contains()`, Java lo busca en el cajón correspondiente al par ("ACC-101", 5).

3. **Fuga de Memoria (Memory Leak):** ¿Por qué este patrón genera una fuga de memoria si se ejecuta repetidamente en un servidor que opera 24/7?

Si agregamos tarjetas continuamente, para luego modificar sus atributos que participan en la creación de sus respectivos `hashCode()`, terminamos con objetos que siguen almacenados dentro del `hashSet` pero que no se pueden encontrar mediante `contains()` o eliminar mediante `remove()`. Si esto ocurre repetidamente en un servidor funcionando 24/7 se acumulan objetos "invisibles" dentro de la colección que siguen ocupando espacio de memoria. Cuando el Garbage

Collector se ejecute para limpiar la memoria, no los eliminará porque el HashSet aún tiene un puntero hacia ellos.

4. **Refactorización Obligatoria:** Aplique dos soluciones arquitectónicas distintas para corregir el problema (una basada en inmutabilidad y otra ajustando el criterio de la entidad).

**Solución basada en inmutabilidad:** No permitir que se realicen cambios sobre los atributos que participan en la generación del hashCode(). Esto es, declarar los atributos *codigo* y *nivelAcceso* como final y eliminar el método *setNivelAcceso()*. Esta solución implica que, si se desea cambiar el nivel de acceso de alguna tarjeta, es obligatorio generar una nueva instancia de *TarjetaAcceso*.

**Solución ajustando el criterio de la identidad:** Modificar los métodos *equals()* y *hashCode()* para que utilicen únicamente el atributo *codigo* al momento de calcular el hashCode. De esta forma, este valor es constante y el objeto no se perderá si se modifica el atributo *nivelAcceso*. Esta solución permite que una misma tarjeta pueda tener distintos niveles de acceso en el tiempo.

## Consigna 2: Selección y Diseño de Arquitectura de Colecciones

### Diseñe el backend de un módulo para una plataforma de E-Commerce

Debe elegir la interfaz y la implementación concreta del Java Collections Framework (java.util) más eficiente para cada una de las siguientes tres funcionalidades:

|                                                                                                                                                                                                                                               |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Requerimiento 1: Sistema de Auditoría de Transacciones</b><br/>Registrar operaciones en el orden exacto en que ocurren. Se requiere insertar miles de registros por segundo al final de la colección y recorrerlos secuencialmente.</p> |
| <p><b>Requerimiento 2: Catálogo de Ofertas Relámpago</b><br/>Mantener los productos ordenados automáticamente por precio de menor a mayor. Debe permitir consultar eficientemente el producto más barato y el más caro.</p>                   |
| <p><b>Requerimiento 3: Búsqueda de Sesiones de Usuarios Activos</b><br/>Buscar instantáneamente la sesión de un usuario dado su Token (String único) entre 500,000 usuarios conectados en simultáneo.</p>                                     |

### Responder y Justificar:

Completar la siguiente tabla comparativa y justificar la decisión con la complejidad algorítmica Big O:

| Requerimiento | Interfaz Seleccionada (List, Set, Map, Queue) | Implementación Concretada (ArrayList, TreeSet, HashMap, etc.) | Complejidad Búsqueda                               | Complejidad Inserción | Justificación Técnica de la Elección                                                                                                                                                                                                                       |
|---------------|-----------------------------------------------|---------------------------------------------------------------|----------------------------------------------------|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Auditoría  | List                                          | ArrayList                                                     | $O(n)$ secuencial<br>$O(1)$ por índice             | $O(1)$                | En una auditoría con miles de registros es importante en qué orden ocurrieron ciertas acciones. Es conveniente utilizar List ya que mantiene el orden. ArrayList puede ir directamente a cualquier posición sin tener que recorrer todas las anteriores.   |
| 2. Ofertas    | Set                                           | TreeSet                                                       | $O(\log n)$ búsqueda<br>$O(1)$ con first(), last() | $O(\log n)$           | TreeSet mantiene las ofertas (precios) ordenados, se puede utilizar first() y last() para encontrar el menor o mayor valor. Para la búsqueda de elementos, basta con hacer una comparación de elementos para decidir si continuar por izquierda o derecha. |
| 3. Sesiones   | Map                                           | HashMap                                                       | $O(1)$                                             | $O(1)$                | HashMap permite relacionar cada Token único con su Sesión, usando el Token como clave para realizar una búsqueda eficiente de la Sesión, evitando tener que buscar entre todas las Sesiones.                                                               |

### Consigna 3: Optimización Algorítmica ( $O(n^2) \rightarrow O(n)$ )

Un desarrollador escribió el siguiente método para encontrar el primer carácter que no se repite dentro de una cadena de texto (ejemplo: en "barco", el primero es 'b'; en "abarca", el primero es 'r').

```
public static Character primerCaracterNoRepetidoLento(String texto) {  
    // Algoritmo ineficiente  $O(n^2)$   
    for (int i = 0; i < texto.length(); i++) {  
        char c = texto.charAt(i);  
        boolean duplicado = false;  
        for (int j = 0; j < texto.length(); j++) {  
            if (i != j && c == texto.charAt(j)) {  
                duplicado = true;  
                break;  
            }  
        }  
        if (!duplicado) return c;  
    }  
    return null;  
}
```

Responder y Justificar:

1. **Refactorización a  $O(n)$ :** Escriba una versión optimizada del método utilizando la colección adecuada de java.util (HashMap, LinkedHashMap, o HashSet) que resuelva el problema realizando como máximo dos recorridos lineales ( $O(n)$ ).

```
public static Character primerCaracterNoRepetido(String texto) {  
    HashMap<Character, Integer> cantidades = new HashMap<>();  
    // primer recorrido: contamos apariciones de cada carácter  
    for (int i = 0; i < texto.length(); i++) {  
        char c = texto.charAt(i);  
        cantidades.put(c, cantidades.getOrDefault(c, 0) + 1);  
    }  
    // segundo recorrido: buscamos el primero que aparezca una vez  
    for (int i = 0; i < texto.length(); i++) {  
        char c = texto.charAt(i);  
        if (cantidades.get(c) == 1) {  
            return c;  
        }  
    }  
    return null;  
}
```

HashMap hace el primer recorrido para contar cuántas veces aparece cada letra en un String. El segundo recorrido verifica si la letra aparece una sola vez, si es así devuelve esa letra. (devuelve el primero cuyo contador sea 1)

2. **Análisis de Colisiones:** Si se utiliza un HashMap internamente, explique qué ocurre si dos caracteres distintos generan el mismo hashCode() (Colisión de Hash) y cómo la estructura resuelve este problema sin perder información.

Si dos caracteres distintos generan el mismo hashCode, el HashMap guardará ambos elementos en el mismo bucket. Cuando se necesite buscar uno de ellos, esta estructura utilizará el método equals() para comparar el elemento buscado con todos los almacenados. De esta manera podrá identificar el correcto sin cometer ningún error.

## Consigna 4: Caso Teórico de Borde y Colisiones

Considere las siguientes dos instancias de String:

- String s1 = "FB";
- String s2 = "Ea";

Sabiendo que `s1.hashCode() == s2.hashCode()` devuelve *true* (ambos dan el entero 2236):

1. Si guardamos ambas cadenas en un HashSet, ¿cuántos elementos contendrá la colección? Explique cómo utiliza Java los métodos hashCode() y equals() internamente para determinar si s2 debe ingresarse o desecharse.

Al insertar *s1*, Java calcula su hashCode correspondiente y lo guarda en su debido bucket. Al insertar *s2*, su hashCode() resultará igual al calculado para *s1* y se producirá una colisión de hash. Para solucionar este problema, Java invoca el método equals() para comparar el objeto nuevo con el existente. Esta operación devolverá *false* porque compara los valores (es decir, "FB y Ea"). De esta manera concluirá que *s2* y *s1* son objetos distintos y conservará ambos.

2. ¿Cómo afecta la acumulación de colisiones de hash al rendimiento teórico de un HashMap en el peor escenario posible (de  $O(1)$  a  $O(n)$  o  $O(\log(n))$ )?

En el peor escenario posible (donde todos los elementos insertados devuelven el mismo hash), se acumulan demasiadas colisiones en un solo cajón.

El rendimiento teórico de búsqueda e inserción se degrada drásticamente desde un estado óptimo  $O(1)$  a uno lineal  $O(n)$  en el momento donde

internamente se forma una lista enlazada larga que hay que recorrer nodo por nodo.

En versiones modernas de Java (a partir de la versión 8), si un cajón supera los 8 elementos colisionados, la lista enlazada se transforma en un Árbol Rojo-Negro, amortiguando la degradación y mejorando el peor caso a  $O(\log n)$ .